

questions

Vladyslav Humeniuk

2026-02-04

part 1

1. Przedstaw i porównaj komputerowe reprezentacje liczb całkowitych i rzeczywistych

W systemach komputerowych liczby są reprezentowane w pamięci w sposób skończony, co wymusza stosowanie różnych formatów dla liczb całkowitych i rzeczywistych. Liczby całkowite (ang. *integers*) są zazwyczaj reprezentowane w systemie stałopozycyjnym, najczęściej w kodzie uzupełnień do dwóch (U2). W tym formacie ustalona liczba bitów (np. 32 lub 64) przeznaczona jest na zapis wartości, przy czym jeden bit (zazwyczaj najstarszy) pełni funkcję bitu znaku. Kluczową cechą tej reprezentacji jest to, że jest ona dokładna – każda liczba całkowita mieszcząca się w zakresie jest przechowywana bez żadnego błędu. Operacje arytmetyczne na liczbach całkowitych są szybkie i precyzyjne, ale zakres liczb jest sztywno ograniczony. Przekroczenie tego zakresu prowadzi do błędu przepełnienia (ang. *overflow*), który polega na “przekręceniu się” licznika (np. dodanie 1 do maksymalnej wartości dodatniej może dać wartość ujemną). Rozkład liczb na osi liczbowej jest równomierny – odległość między sąsiednimi liczbami całkowitymi wynosi zawsze 1.

Liczby rzeczywiste są reprezentowane w systemie zmiennoprzecinkowym (ang. *floating point*), zdefiniowanym standardem IEEE 754. Liczba x jest zapisywana jako trójka: znak s , mantysa m oraz cecha (wykładnik) c , zgodnie ze wzorem:

$$x = (-1)^s \cdot m \cdot 2^{c-\text{bias}}$$

W przeciwieństwie do liczb całkowitych, reprezentacja zmiennoprzecinkowa nie jest dokładna dla wszystkich liczb rzeczywistych, lecz jest jedynie ich przybliżeniem. Zamiast stałego zakresu, format ten oferuje olbrzymią dynamikę (zakres wartości od bardzo małych do bardzo dużych), ale kosztem stałej precyzji względnej, a nie bezwzględnej. Oznacza to, że liczby nie są rozmieszczone równomiernie na osi: w okolicy zera są one bardzo gęste, a dla dużych wartości odległości między sąsiednimi liczbami mogą być ogromne. Podstawową różnicą jest więc to, że typ całkowity gwarantuje dokładność w małym zakresie, podczas gdy typ rzeczywisty oferuje duży zakres kosztem błędów zaokrągleń.

2. Opisz własności numeryczne operacji zmiennoprzecinkowych

Arytmetyka zmiennoprzecinkowa różni się fundamentalnie od arytmetyki liczb rzeczywistych znanej z matematyki teoretycznej, głównie z powodu skończonej precyzji reprezentacji. Podstawową własnością jest występowanie błędu zaokrąglenia przy niemal każdej operacji. Wynik działania (np. dodawania czy mnożenia) dwóch liczb maszynowych często nie jest liczbą maszynową i musi zostać zaokrąglony do najbliższej wartości reprezentowalnej. Prowadzi to do tego, że podstawowe prawa arytmetyki, takie jak łączność dodawania, przestają obowiązywać. W ogólności:

$$(a + b) + c \neq a + (b + c)$$

Różnica ta jest szczególnie widoczna, gdy dodajemy do siebie liczby o bardzo różnych rzędach wielkości – mała liczba dodana do dużej może zostać “wchłonięta” i całkowicie zignorowana w wyniku zaokrąglenia (zjawisko *absorpcji*).

Kolejną krytyczną własnością jest zjawisko redukcji cyfr znaczących (ang. *catastrophic cancellation*). Występuje ono podczas odejmowania od siebie dwóch bardzo bliskich sobie liczb obarczonych błędem. W wyniku takiej operacji najbardziej znaczące cyfry znoszą się, a w wyniku pozostają jedynie cyfry, które w oryginale były na pozycjach najmniej znaczących i często są wynikiem błędów (szumem numerycznym). Prowadzi to do

gwałtownej utraty dokładności względnej wyniku. Ponadto operacje zmiennoprzecinkowe mogą prowadzić do dwóch typów błędów zakresu: nadmiaru (ang. *overflow*), gdy wynik jest zbyt duży, by go zapisać (co daje nieskończoność), oraz niedomiaru (ang. *underflow*), gdy wynik jest zbyt bliski zera, by go odróżnić od zera przy danej precyzji. Wszystkie te własności wymuszają na programistach i matematykach stosowanie specjalnych algorytmów, które są numerycznie stabilne, czyli odporne na kumulację tych błędów.

3. Podaj definicje i objaśnij na przykładach pojęcia: zadanie, algorytm, realizacja zmiennoprzecinkowa algorytmu

Rozróżnienie między tymi trzema pojęciami jest kluczowe dla zrozumienia numeryki. **Zadanie obliczeniowe** to matematycznie sformułowany problem, który chcemy rozwiązać. Określa ono relację między danymi wejściowymi a wynikami, zakładając istnienie i jednoznaczność rozwiązania w dziedzinie liczb rzeczywistych. Przykładem zadania jest znalezienie pierwiastka równania $x^2 - 2 = 0$. Wynikiem tego zadania jest liczba niewymierna $\sqrt{2}$, która ma nieskończone rozwinięcie dziesiętne.

Algorytm to skończony ciąg jednoznacznie zdefiniowanych operacji logicznych i matematycznych, które prowadzą do rozwiązania zadania. Algorytm operuje na idealnych liczbach rzeczywistych i teoretycznie powinien dać dokładny wynik (lub ciąg zbieżny do wyniku). Przykładem algorytmu dla powyższego zadania jest metoda Newtona, zdefiniowana wzorem $x_{n+1} = \frac{1}{2}(x_n + \frac{2}{x_n})$. W teorii, po nieskończonej liczbie kroków, ciąg ten zbiega do $\sqrt{2}$.

Realizacja zmiennoprzecinkowa algorytmu to implementacja tego algorytmu w konkretnym środowisku komputerowym, przy użyciu arytmetyki o skończonej precyzji (np. standard IEEE 754). W tym przypadku każda operacja matematyczna jest zastępowana jej przybliżeniem maszynowym, obciążonym błędem zaokrąglenia. Realizacja nie operuje na liczbach rzeczywistych, lecz na ich maszynowych reprezentacjach. W naszym przykładzie, komputer nie obliczy $\sqrt{2}$, lecz zwróci wartość przybliżoną, np. 1.41421356, a dalsze iteracje algorytmu Newtona przestaną poprawiać wynik z powodu osiągnięcia precyzji maszynowej.

4. Podaj definicje i objaśnij na przykładach pojęcia: uwarunkowanie zadania, poprawność numeryczna algorytmu, stabilność numeryczna algorytmu

Uwarunkowanie zadania (ang. *conditioning*) to cecha samego problemu obliczeniowego, niezależna od metody jego rozwiązywania. Określa ona, jak bardzo wynik końcowy jest wrażliwy na niewielkie zmiany (zaburzenia) danych wejściowych. Jeśli mała zmiana danych wejściowych powoduje małą zmianę wyniku, zadanie jest dobrze uwarunkowane. Jeśli natomiast małe zaburzenie na wejściu prowadzi do ogromnych zmian na wyjściu, zadanie jest źle uwarunkowane. Miarą tej wrażliwości jest wskaźnik uwarunkowania. Klasycznym przykładem źle uwarunkowanego zadania jest wielomian Wilkinsona: znalezienie pierwiastków wielomianu wysokiego stopnia na podstawie jego współczynników. Minimalna zmiana jednego współczynnika może drastycznie zmienić wartości pierwiastków.

Poprawność numeryczna algorytmu oznacza, że algorytm realizuje zadanie z możliwie największą dokładnością, na jaką pozwala użyta arytmetyka (np. podwójnej precyzji). Algorytm jest poprawny numerycznie, jeśli błąd ostatecznego wyniku jest rzędu błędu wynikającego z samego uwarunkowania zadania i precyzji arytmetyki. Oznacza to, że algorytm nie wprowadza dodatkowych, systematycznych błędów, które fałszowałyby wynik bardziej niż jest to nieuniknione.

Stabilność numeryczna algorytmu to cecha konkretnej metody obliczeniowej. Algorytm nazywamy numerycznie stabilnym, jeśli błędy zaokrągleń popełniane w trakcie obliczeń nie kumulują się w sposób niekontrolowany i nie powodują drastycznego zafałszowania wyniku końcowego. Formalnie, stabilność wsteczna oznacza, że wynik obliczony przez algorytm jest dokładnym rozwiązaniem dla pewnych, nieco zaburzonych danych wejściowych. Przykładem niestabilnego algorytmu jest wyznaczanie pochodnej funkcji z definicji ilorazu różnicowego dla bardzo małego kroku h – błędy zaokrągleń przy odejmowaniu bliskich wartości $f(x+h)$ i $f(x)$ dominują nad wynikiem. Algorytm stabilny w tej sytuacji zastosowałby inną metodę, np. różniczkowanie analityczne lub specjalne wzory o wyższej dokładności.

5. Porównaj interpolację Lagrange’a i interpolację Hermite’a

Interpolacja wielomianowa to proces wyznaczania wielomianu, który przyjmuje zadane wartości w określonych punktach węzłowych. Interpolacja Lagrange’a i Hermite’a to dwa różne podejścia do tego problemu, różniące się ilością i rodzajem wykorzystywanych informacji o funkcji. W interpolacji Lagrange’a dysponujemy jedynie zbiorem $n + 1$ węzłów x_i oraz odpowiadających im wartości funkcji y_i . Zadaniem jest znalezienie wielomianu stopnia co najwyżej n , który przechodzi przez wszystkie te punkty. Wielomian ten jest wyznaczany jednoznacznie i można go zapisać w postaci sumy ważonej wielomianów bazowych:

$$L_n(x) = \sum_{i=0}^n y_i \prod_{j=0, j \neq i}^n \frac{x - x_j}{x_i - x_j}$$

Metoda ta zapewnia zgodność wartości wielomianu z wartościami funkcji w węzłach, ale nie gwarantuje zgodności kształtu krzywej (np. kierunku stycznej) pomiędzy węzłami.

Interpolacja Hermite’a jest uogólnieniem interpolacji Lagrange’a, w której oprócz wartości funkcji w węzłach y_i , wykorzystujemy również informacje o wartościach jej pochodnych y'_i (a czasem także wyższych pochodnych) w tych samych punktach. Jeśli dla każdego z $n + 1$ węzłów zadana jest wartość funkcji oraz jej pierwszej pochodnej, poszukujemy wielomianu stopnia co najwyżej $2n + 1$. Wielomian Hermite’a nie tylko przecina punkty pomiarowe, ale jest również styczny do interpolowanej funkcji w każdym z tych punktów. Dzięki temu krzywa interpolacyjna znacznie wierniej odwzorowuje przebieg oryginalnej funkcji, jest gładzsza i zazwyczaj obciążona mniejszym błędem interpolacji. Wadą metody Hermite’a jest konieczność znajomości pochodnych, co w przypadku danych eksperymentalnych często jest niemożliwe.

6. Objaśnij efekt Rungego - jak się objawia, jakie są jego przyczyny

Efekt Rungego to zjawisko pogorszenia się jakości interpolacji wielomianowej, które występuje przy stosowaniu wielomianów wysokiego stopnia na równomiernie rozmieszczonych węzłach. Objawia się ono występowaniem silnych oscylacji wielomianu interpolacyjnego w pobliżu krańców przedziału interpolacji, mimo że w środku przedziału przybliżenie może być bardzo dokładne. Klasycznym przykładem ilustrującym to zjawisko jest interpolacja funkcji $f(x) = \frac{1}{1+25x^2}$ na przedziale $[-1, 1]$. Wraz ze wzrostem liczby węzłów (a tym samym stopnia wielomianu), maksymalny błąd interpolacji zamiast maleć, rośnie do nieskończoności.

Przyczyną efektu Rungego jest natura wielomianów wysokiego stopnia oraz równomierne rozmieszczenie węzłów. Błąd interpolacji w punkcie x wyraża się wzorem zawierającym iloczyn $(x - x_0)(x - x_1) \dots (x - x_n)$. W przypadku węzłów równoodległych, wartość tego iloczynu drastycznie rośnie w pobliżu krańców przedziału. Wielomian, próbując “trafić” we wszystkie węzły rozmieszczone sztywno w równych odstępach, musi wykonywać gwałtowne “zwroty”, co prowadzi do nienaturalnych pofalowań (oscylacji). Aby uniknąć tego efektu, nie należy zwiększać stopnia wielomianu przy węzłach równoodległych. Rozwiązaniem jest zastosowanie węzłów Czebyszewa (które są zagęszczane na krańcach przedziału) lub rezygnacja z jednego wielomianu wysokiego stopnia na rzecz funkcji sklepanych.

7. Podaj definicję funkcji sklepanych, wyjaśnij ją na rysunku; wyjaśnij dlaczego funkcje sklepane są użyteczne

Funkcja sklejana (ang. *spline*) stopnia k to funkcja, która na całym przedziale interpolacji $[a, b]$ jest sklejana z kawałków wielomianów stopnia co najwyżej k . Cały przedział podzielony jest na podprzedziały za pomocą węzłów $x_0, x_1, \dots, x_n$. Wewnątrz każdego podprzedziału $[x_i, x_{i+1}]$ funkcja jest wielomianem, a w samych węzłach spełnione są warunki ciągłości nie tylko samej funkcji, ale również jej pochodnych do rzędu $k - 1$ włącznie. Najczęściej stosuje się funkcje sklepane trzeciego stopnia (kubiczne), które składają się z wielomianów trzeciego stopnia i zapewniają ciągłość funkcji, jej pierwszej i drugiej pochodnej w każdym węźle (klasa gładkości C^2).

Funkcje sklepane są niezwykle użyteczne w numeryce i grafice komputerowej, ponieważ eliminują wady interpolacji wielomianami wysokiego stopnia, w tym wspomniany wcześniej efekt Rungego. Dzięki temu, że stopień wielomianów jest niski (zazwyczaj 3), funkcja nie wpada w oscylacje i zachowuje się stabilnie numerycznie. Jednocześnie, dzięki warunkom ciągłości pochodnych, wykres funkcji jest gładki i nie posiada

“kantów” w punktach łączenia (węzłach). Splajny sześciennie mają również własność minimalizacji energii krzywizny – zachowują się jak elastyczna listwa przechodząca przez zadane punkty, co daje bardzo naturalny wygląd krzywej.

8. Porównaj interpolację wielomianową i funkcjami sklejanymi

Interpolacja wielomianowa (globalna) polega na znalezieniu jednego, unikalnego wielomianu $P_n(x)$, który opisuje funkcję na całym badanym przedziale. Interpolacja funkcjami sklejanymi (lokalna) dzieli przedział na mniejsze segmenty i dla każdego z nich definiuje osobny wielomian niskiego stopnia. Podstawową różnicą jest zachowanie przy dużej liczbie węzłów. W interpolacji wielomianowej zwiększanie liczby węzłów wymusza podnoszenie stopnia wielomianu, co prowadzi do niestabilności numerycznej, kosztownych obliczeń i efektu Rungego (oscylacji na krańcach). W przypadku funkcji sklepanych zwiększenie liczby węzłów powoduje jedynie dodanie kolejnych segmentów wielomianowych niskiego stopnia, bez pogarszania jakości przybliżenia w innych miejscach.

Pod względem analitycznym, interpolacja wielomianowa daje prostszy, zwarty wzór (jeden wielomian), który łatwo różniczkować i całkować analitycznie w całym przedziale. Funkcje sklepane mają postać złożoną (wiele wzorów definiowanych przedziałami), co może być trudniejsze w analizie teoretycznej, ale jest znacznie bardziej efektywne w implementacji komputerowej. W praktyce inżynierskiej, funkcje sklepane są preferowane do wizualizacji danych i modelowania kształtów, podczas gdy interpolacja wielomianowa (niskiego stopnia) jest stosowana lokalnie lub w specyficznych metodach teoretycznych, np. w kwadraturach.

9. Porównaj aproksymację średniokwadratową i jednostajną, wyjaśnij kiedy każda z nich jest użyteczna

Aproksymacja to proces przybliżania nieznanej lub skomplikowanej funkcji $f(x)$ inną, prostszą funkcją $g(x)$, przy czym – w odróżnieniu od interpolacji – funkcja przybliżająca nie musi przechodzić dokładnie przez punkty pomiarowe. Różnica między aproksymacją średniokwadratową a jednostajną leży w sposobie definiowania błędu (normy), który staramy się zminimalizować.

Aproksymacja średniokwadratowa minimalizuje sumę kwadratów różnic między wartościami funkcji a aproksymacją w punktach pomiarowych (lub całkę z kwadratu różnicy w przypadku ciągłym):

$$S = \sum (f(x_i) - g(x_i))^2 \rightarrow \min$$

Jest ona najbardziej użyteczna przy analizie danych eksperymentalnych, obarczonych błędami pomiarowymi (szumem). Metoda ta pozwala “wygładzić” dane, ignorując pojedyncze, mocno odstające punkty, które mogłyby być wynikiem błędu grubego. Traktuje ona błędy jako zmienne losowe i znajduje trend danych.

Aproksymacja jednostajna (Czebyszewa) minimalizuje maksymalny błąd bezwzględny na całym przedziale:

$$E = \max |f(x) - g(x)| \rightarrow \min$$

Jest ona użyteczna w sytuacjach, gdzie musimy zagwarantować, że błąd w żadnym punkcie nie przekroczy pewnej krytycznej wartości (np. określonej tolerancji błędu). Typowym zastosowaniem jest implementacja funkcji matematycznych (np. `sin`, `exp`) w bibliotekach komputerowych lub kalkulatorach, gdzie zależy nam na tym, by wynik był wiarygodny dla każdego argumentu, a nie tylko “średnio” poprawny. Aproksymacja ta jest jednak trudniejsza obliczeniowo do wyznaczenia (algorytm Remez) i bardzo wrażliwa na punkty odstające w danych.

10. Opisz znane Ci metody aproksymacji jednostajnej

Aproksymacja jednostajna polega na znalezieniu wielomianu $W_n(x)$, który minimalizuje maksymalny błąd bezwzględny względem funkcji aproksymowanej $f(x)$ na przedziale $[a, b]$. Oznacza to minimalizację normy Czebyszewa: $\|f - W_n\|_\infty = \max_{x \in [a, b]} |f(x) - W_n(x)|$. Najważniejszą metodą w tej dziedzinie jest konstrukcja oparta na **wielomianach Czebyszewa**. Jeśli węzły interpolacji wybierzemy w miejscach zerowych wielomianów Czebyszewa (zamiast węzłów równoodległych), otrzymany wielomian interpolacyjny jest bliski optymalnemu wielomianowi aproksymującemu w sensie jednostajnym.

Dokładne rozwiązanie problemu najlepszej aproksymacji jednostajnej opisuje **twierdzenie o alternansie**. Mówi ono, że wielomian $W_n(x)$ jest najlepszym przybliżeniem funkcji $f(x)$ wtedy i tylko wtedy, gdy funkcja błędu $e(x) = f(x) - W_n(x)$ osiąga swoją maksymalną wartość bezwzględną w co najmniej $n + 2$ punktach przedziału, z naprzemiennymi znakami błędu (plus, minus, plus...). Do numerycznego znalezienia takiego wielomianu stosuje się **algorytm Remeza**. Jest to algorytm iteracyjny, który startuje od pewnego zbioru punktów, a następnie w każdym kroku przesuwa je tak, aby wyrównać ekstrema funkcji błędu, aż do spełnienia warunku alternansu.

11. Porównaj kwadratury Newtona-Cotesa i Gaussa

Obie metody służą do numerycznego obliczania całek oznaczonych $\int_a^b f(x)dx$ jako sumy ważonej wartości funkcji w wybranych punktach (węzłach). Różnica polega na sposobie wyboru tych węzłów i wag. **Kwadratury Newtona-Cotesa** wykorzystują węzły rozmieszczone równomiernie (ekwidystantne) na przedziale całkowania. Przykładami są metoda trapezów (2 węzły na krańcach) i metoda Simpsona (3 węzły). Są one proste w implementacji i intuicyjne, ale przy wysokich rzędach stają się numerycznie niestabilne (niektóre wagi stają się ujemne) i podlegają efektowi Rungego. Dokładnie całkują wielomiany stopnia do n (dla n nieparzystych) lub $n + 1$ (dla n parzystych).

Kwadratury Gaussa rezygnują z równomiernego rozmieszczenia węzłów na rzecz maksymalizacji dokładności. Węzły i wagi są dobierane w sposób optymalny – węzły są pierwiastkami pewnych wielomianów ortogonalnych (np. wielomianów Legendre’a dla przedziału $[-1, 1]$). Dzięki temu kwadratura Gaussa wykorzystująca n węzłów potrafi dokładnie obliczyć całkę dla wielomianu stopnia aż $2n - 1$. Oznacza to, że kwadratury Gaussa są znacznie bardziej efektywne – wymagają znacznie mniejszej liczby obliczeń wartości funkcji, aby uzyskać tę samą dokładność, co metody Newtona-Cotesa. Ich wadą jest konieczność przechowywania tablic ze specyficznymi wartościami węzłów i wag oraz trudniejsza adaptacja do danych empirycznych, które są już „sztywno” zebrane w równych odstępach.

12. Wyjaśnij istotę algorytmu całkowania adaptacyjnego

Standardowe metody całkowania (np. złożona metoda Simpsona) dzielą przedział całkowania na podprzedziały o stałej długości h . Jest to nieefektywne, jeśli funkcja $f(x)$ w jednej części przedziału zmienia się bardzo wolno (tam wystarczyłby duży krok), a w innej oscyluje bardzo gwałtownie (tam potrzebny jest bardzo mały krok). Całkowanie adaptacyjne automatycznie dostosowuje gęstość próbkowania do zachowania funkcji, aby utrzymać zadany błąd przy minimalnym koszcie obliczeniowym.

Algorytm działa zazwyczaj w sposób rekurencyjny. Na danym przedziale $[a, b]$ całka obliczana jest dwukrotnie: raz z krokiem h (wynik I_1), a drugi raz z krokiem $h/2$ (wynik I_2). Różnica $|I_2 - I_1|$ stanowi oszacowanie błędu na tym odcinku. Jeśli błąd jest mniejszy od założonej tolerancji ϵ , wynik I_2 jest akceptowany. Jeśli błąd jest zbyt duży, przedział $[a, b]$ jest dzielony na dwie połowy, a procedura jest wywoływana rekurencyjnie dla każdej z nich (z odpowiednio zmniejszoną tolerancją, np. $\epsilon/2$). Dzięki temu algorytm „zagęszcza” siatkę punktów tylko tam, gdzie funkcja jest trudna do scałkowania, oszczędzając czas w obszarach „łatwych”.

13. Wyjaśnij na czym polega iteracyjne rozwiązywanie równań nieliniowych; podaj warunek zbieżności

Rozwiązywanie równań nieliniowych metodami iteracyjnymi polega na generowaniu ciągu przybliżeń rozwiązania, który z każdym kolejnym krokiem zbliża się do poszukiwanego pierwiastka funkcji. Zamiast próbować obliczyć wynik w jednym kroku (co dla wielu równań, jak np. $x = \cos(x)$, jest niemożliwe analitycznie), zaczynamy od punktu startowego x_0 (lub przedziału $[a, b]$) i stosujemy określony wzór rekurencyjny, aby uzyskać $x_1, x_2, \dots, x_k$. Proces ten powtarzamy aż do momentu, gdy różnica między kolejnymi przybliżeniami $|x_{k+1} - x_k|$ będzie mniejsza od założonej dokładności (tolerancji błędu) lub gdy wartość funkcji $|f(x_k)|$ będzie wystarczająco bliska zeru. Istotą tego procesu jest to, że wynik nigdy nie jest „dokładny” w sensie matematycznym, ale jest „wystarczająco dokładny” do zastosowań inżynierskich.

Warunkiem koniecznym zbieżności metody iteracyjnej zapisanym w postaci ogólnej $x_{k+1} = \Phi(x_k)$ jest tak zwany warunek zwężający odwzorowania. Aby ciąg x_k zbiegał do rozwiązania dokładnego x^* , pochodna funkcji

iteracyjnej Φ w otoczeniu pierwiastka musi spełniać nierówność $|\Phi'(x)| < 1$. Oznacza to, że z każdym krokiem błąd przybliżenia musi maleć. Jeśli wartość bezwzględna pochodnej jest mniejsza od 1, to metoda będzie zbieżna lokalnie, czyli przy odpowiednio dobrym przybliżeniu początkowym ciąg iteracji doprowadzi nas do rozwiązania.

14. Przedstaw metodę Newtona-Raphsona rozwiązywania równań nieliniowych

Metoda Newtona-Raphsona, zwana również metodą stycznych, jest jednym z najszybszych i najczęściej stosowanych algorytmów iteracyjnych służących do znajdowania pierwiastków funkcji rzeczywistej. Idea geometryczna tej metody polega na przybliżaniu wykresu funkcji nieliniowej za pomocą linii stycznej. W każdym kroku iteracji, zamiast szukać miejsca zerowego samej funkcji (co jest trudne), znajdujemy miejsce zerowe stycznej do wykresu tej funkcji poprowadzonej w bieżącym punkcie przybliżenia.

Proces rozpoczynamy od wyboru punktu startowego x_0 , który powinien znajdować się blisko poszukiwanego pierwiastka. Następnie wyznaczamy równanie stycznej do krzywej $y = f(x)$ w punkcie $(x_0, f(x_0))$. Punkt przecięcia tej stycznej z osią odciętych staje się nowym przybliżeniem x_1 . Powtarzając ten proces, generujemy ciąg przybliżeń zdefiniowany wzorem rekurencyjnym:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Kluczową cechą metody Newtona jest jej zbieżność kwadratowa, co oznacza podwajanie liczby cyfr dokładnych w każdym kroku. Metoda wymaga jednak znajomości pochodnej $f'(x)$ i dobrego punktu startowego.

15. Opisz interpolacyjne metody rozwiązywania równań nieliniowych

Interpolacyjne metody rozwiązywania równań nieliniowych opierają się na zastąpieniu skomplikowanej funkcji nieliniowej prostszą funkcją, zazwyczaj wielomianem niskiego stopnia, na pewnym lokalnym przedziale. Najczęściej stosowanym wariantem jest interpolacja liniowa, która prowadzi do metody siecznych (ang. Secant Method) oraz metody *regula falsi*. W metodzie siecznych rezygnujemy z konieczności obliczania pochodnej funkcji. Zamiast używać stycznej, przybliżamy funkcję prostą sieczną przechodzącą przez dwa ostatnie punkty iteracji: $(x_{n-1}, f(x_{n-1}))$ oraz $(x_n, f(x_n))$. Nowym przybliżeniem pierwiastka staje się punkt przecięcia tej siecznej z osią X .

Wzór iteracyjny metody siecznych przyjmuje postać:

$$x_{n+1} = x_n - f(x_n) \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}$$

Metoda ta jest metodą dwupunktową i ma rząd zbieżności około 1.618 (złota liczba). Inną metodą interpolacyjną jest metoda Müllera, która wykorzystuje interpolację kwadratową (parabolę przechodzącą przez trzy punkty), co pozwala na znajdowanie pierwiastków zespolonych i często zapewnia szybszą zbieżność niż metody liniowe.

16. Scharakteryzuj trudności rozwiązywania układów równań nieliniowych

Rozwiązywanie układów równań nieliniowych (postaci $F(x) = 0$, gdzie x jest wektorem) jest problemem znacznie trudniejszym niż w przypadku pojedynczego równania czy układu liniowego. Podstawową trudnością jest brak ogólnych metod gwarantujących znalezienie rozwiązania dla dowolnego układu. W przeciwieństwie do równań liniowych, nie mamy prostych metod bezpośrednich (jak eliminacja Gaussa). Musimy polegać na metodach iteracyjnych, które wymagają podania punktu startowego x_0 . Wybór tego punktu jest krytyczny – jeśli jest zbyt daleko od rozwiązania, metoda (np. Newtona dla układów) może być rozbieżna.

Kolejną trudnością jest koszt obliczeniowy. Metoda Newtona dla układów wymaga w każdym kroku obliczenia macierzy Jacobiego (macierzy wszystkich pochodnych cząstkowych), a następnie rozwiązania układu równań liniowych z tą macierzą. Dla układu N równań ma to złożoność rzędu $O(N^3)$ w każdej iteracji, co dla dużych układów jest zaporowe. Ponadto, macierz Jacobiego może stać się osobiwa w pobliżu rozwiązania, co uniemożliwia wykonanie kroku. Dodatkowym problemem jest niejednoznaczność rozwiązań – układ nieliniowy może mieć jedno rozwiązanie, wiele rozwiązań lub nie mieć ich wcale, a algorytm numeryczny zazwyczaj znajduje tylko jedno z nich, bez informacji o istnieniu pozostałych.

17. Podaj algorytm rozwiązywania układów równań liniowych metodą eliminacji Gaussa

Metoda eliminacji Gaussa to algorytm służący do rozwiązywania układów równań liniowych postaci $Ax = b$, gdzie A jest macierzą kwadratową współczynników, x wektorem niewiadomych, a b wektorem wyrazów wolnych. Algorytm ten składa się z dwóch głównych etapów: eliminacji w przód oraz podstawiania wstecz. Celem pierwszego etapu jest przekształcenie układu do równoważnej postaci, w której macierz współczynników staje się macierzą trójkątną górną (wszystkie elementy pod główną przekątną są zerami).

Etap 1: Eliminacja w przód. Proces przebiega w $n - 1$ krokach dla macierzy o wymiarze $n \times n$. W k -tym kroku wybieramy element główny a_{kk} (tzw. pivot) i używamy go do wyzerowania wszystkich elementów znajdujących się pod nim w tej samej kolumnie. Operacja ta polega na odjęciu od i -tego wiersza (gdzie $i > k$) wiersza k -tego pomnożonego przez współczynnik:

$$m_{ik} = \frac{a_{ik}}{a_{kk}}$$

Tę samą operację wykonujemy na wektorze b . Aby zapewnić stabilność numeryczną i uniknąć dzielenia przez zero, stosuje się wybór elementu podstawowego (pivoting), czyli zamianę wierszy tak, aby na przekątnej znalazł się element o największym module w danej kolumnie.

Etap 2: Podstawianie wstecz. Po zakończeniu eliminacji otrzymujemy macierz trójkątną górną U . Ostatnie równanie w układzie zawiera już tylko jedną niewiadomą x_n , którą możemy łatwo wyznaczyć. Następnie, znając x_n , przechodzimy do przedostatniego równania, aby wyznaczyć x_{n-1} , i kontynuujemy ten proces w górę, aż do wyznaczenia x_1 . Ogólny wzór na wyznaczenie niewiadomej x_i w tym etapie to:

$$x_i = \frac{1}{u_{ii}} \left(b'_i - \sum_{j=i+1}^n u_{ij}x_j \right)$$

Złożoność obliczeniowa całego algorytmu wynosi $O(n^3)$, co czyni go skutecznym dla układów o małych i średnich rozmiarach.

18. Objaśnij na czym polega przewaga algorytmów faktoryzacji LU nad metodą eliminacji Gaussa

Faktoryzacja LU jest metodą rozkładu macierzy A na iloczyn dwóch macierzy trójkątnych: dolnotrójkątnej L (Lower) i górnortrójkątnej U (Upper), tak że $A = LU$. Metoda eliminacji Gaussa w swojej klasycznej formie wykonuje redukcję macierzy rozszerzonej $[A|b]$ jednocześnie dla współczynników i wyrazów wolnych. Przewaga LU ujawnia się, gdy musimy rozwiązać wiele układów równań $Ax = b$ z tą samą macierzą A , ale różnymi wektorami b (np. w symulacjach, gdzie A opisuje geometrię układu, a b zmieniające się siły).

W metodzie Gaussa dla każdego nowego b musielibyśmy powtarzać cały proces eliminacji, którego koszt wynosi $O(n^3)$. W podejściu LU, kosztowny rozkład macierzy A na czynniki L i U (koszt $O(n^3)$) wykonujemy tylko raz. Następnie, dla każdego nowego wektora b , rozwiązujemy układ w dwóch szybkich krokach: $Ly = b$ (podstawianie w przód) oraz $Ux = y$ (podstawianie wstecz). Oba te kroki mają koszt tylko $O(n^2)$. Zatem przy wielokrotnym rozwiązywaniu układu, faktoryzacja LU daje ogromną oszczędność czasu obliczeniowego.

19. Wyjaśnij dlaczego i w jakich przypadkach należy używać iteracyjnych metod rozwiązywania układów równań liniowych

Metody bezpośrednie, takie jak eliminacja Gaussa czy faktoryzacja LU, dają (teoretycznie) dokładny wynik po skończonej liczbie kroków, ale mają złożoność czasową $O(n^3)$ i pamięciową $O(n^2)$. Stają się one nieefektywne lub wręcz niemożliwe do użycia, gdy mamy do czynienia z układami o bardzo dużych rozmiarach (np. $n > 100,000$), które są macierzami rzadkimi (ang. *sparse matrices*). Macierze rzadkie zawierają głównie zera – np. w metodzie elementów skończonych (MES) każdy element oddziałuje tylko z kilkoma sąsiadami.

Podczas faktoryzacji LU macierzy rzadkiej następuje zjawisko “fill-in” – zera zamieniają się w wartości niezerowe, co drastycznie zwiększa zapotrzebowanie na pamięć RAM. Metody iteracyjne (np. Jacobiego, CG, GMRES) nie modyfikują macierzy A , a jedynie wykonują operacje mnożenia macierzy przez wektor. Dzięki

temu mogą one wykorzystać strukturę rzadką macierzy, przechowując tylko niezerowe elementy. Pozwala to na rozwiązywanie gigantycznych układów równań przy złożoności czasowej bliskiej $O(n)$ na iterację i minimalnym zużyciu pamięci. Używamy ich więc zawsze wtedy, gdy macierz jest duża i rzadka, lub gdy wystarcza nam przybliżone rozwiązanie uzyskane w krótkim czasie.

20. Porównaj metody iteracyjne rozwiązywania układów równań liniowych: Jacobiego, S-R, SOR, Czebyszewa

Metoda Jacobiego jest najprostszą metodą iteracyjną. Polega na wyznaczeniu i -tej niewiadomej z i -tego równania, przy założeniu, że pozostałe niewiadome są znane z poprzedniej iteracji. Jej główną zaletą jest możliwość łatwego zrównoleglenia (wszystkie nowe wartości $x^{(k+1)}$ zależą tylko od starych $x^{(k)}$), ale zbieżność jest zazwyczaj bardzo wolna i gwarantowana tylko dla macierzy diagonalnie dominujących.

Metoda Gaussa-Seidla (G-S) (w pytaniu oznaczona jako S-R, co może być błędem w nazewnictwie lub skrótem od “Successive Relaxation” bez parametru) jest ulepszeniem metody Jacobiego. W iteracji $k + 1$ do obliczenia kolejnych niewiadomych wykorzystuje się już te wartości $x_j^{(k+1)}$, które zostały obliczone w bieżącym kroku. Dzięki temu informacja propaguje się szybciej, co zazwyczaj daje lepszą zbieżność niż metoda Jacobiego, ale utrudnia zrównoleglenie algorytmu.

Metoda SOR (Successive Over-Relaxation) to modyfikacja metody Gaussa-Seidla, która wprowadza parametr relaksacji ω . Nowe przybliżenie jest średnią ważoną starej wartości i wartości wyliczonej metodą G-S. Odpowiedni dobór parametru ω (zazwyczaj $1 < \omega < 2$) pozwala na “przeszacowanie” zmiany, co drastycznie przyspiesza zbieżność metody. SOR jest jedną z najefektywniejszych prostych metod iteracyjnych.

Metoda Czebyszewa (lub przyspieszenie Czebyszewa) to technika, która nie jest samodzielną metodą stacjonarną jak powyższe, lecz metodą niestacjonarną, gdzie parametry iteracji zmieniają się w każdym kroku. Wykorzystuje ona własności wielomianów Czebyszewa do minimalizacji błędu w oparciu o wiedzę o spektrum (wartościach własnych) macierzy. Jest szybsza od SOR w pewnych warunkach, ale wymaga oszacowania minimalnej i maksymalnej wartości własnej macierzy, co bywa kosztowne.

21. Wyjaśnij dlaczego istotnym krokiem każdej metody rozwiązywania układów równań liniowych jest szukanie elementu wiodącego (pivoting)

W algorytmach eliminacji (np. Gaussa) kluczową operacją jest dzielenie przez element leżący na głównej przekątnej (tzw. pivot) w celu wyzerowania kolumny pod nim. Jeśli ten element jest równy zero, dzielenie jest niemożliwe i algorytm załamuje się. Co ważniejsze, jeśli element ten jest bardzo bliski zera (mały co do modułu), dzielenie przez niego powoduje powstanie bardzo dużych liczb.

W arytmetyce zmiennoprzecinkowej operacje na liczbach o drastycznie różnych rzędach wielkości prowadzą do ogromnych błędów zaokrągleń i utraty cyfr znaczących. Niestabilność ta może sprawić, że wynik końcowy będzie całkowicie błędny (np. różny od rzeczywistego o rzędy wielkości). Wybór elementu wiodącego (pivoting) polega na zamianie wierszy (pivoting częściowy) lub wierszy i kolumn (pivoting pełny) tak, aby na pozycji dzielnika znalazł się element o największym możliwym module. Gwarantuje to, że mnożniki używane w eliminacji są ograniczone (mniejsze od 1), co powstrzymuje wzrost błędów numerycznych i zapewnia stabilność algorytmu.

22. Omów bibliotekę BLAS i różnice między level 1, 2, 3

BLAS (Basic Linear Algebra Subprograms) to standardowy interfejs programistyczny (API) dla podstawowych operacji algebry liniowej. Jest to de facto standard przemysłowy; biblioteki zoptymalizowane pod konkretny sprzęt (np. Intel MKL, OpenBLAS) implementują ten interfejs, zapewniając maksymalną wydajność obliczeń. Funkcje BLAS dzielą się na trzy poziomy (levels) w zależności od złożoności operacji i stosunku liczby obliczeń do ilości przesyłanych danych.

Level 1 obejmuje operacje wektorowe, takie jak dodawanie wektorów ($y = \alpha x + y$), iloczyn skalarny czy norma wektora. Złożoność obliczeniowa wynosi $O(n)$, a ilość danych $O(n)$. Stosunek obliczeń do dostępu do pamięci jest niski, więc wydajność jest ograniczona przepustowością pamięci RAM.

Level 2 to operacje macierz-wektor, np. mnożenie macierzy przez wektor ($y = Ax + y$) czy rozwiązywanie układów trójkątnych. Złożoność obliczeniowa wynosi $O(n^2)$, a ilość danych również $O(n^2)$. Wydajność jest lepsza niż w Level 1, ale wciąż mocno zależna od pamięci.

Level 3 zawiera operacje macierz-macierz, z których najważniejszą jest mnożenie macierzy ($C = \alpha AB + \beta C$). Złożoność obliczeniowa wynosi $O(n^3)$, podczas gdy ilość danych to tylko $O(n^2)$. Jest to kluczowa różnica: wykonujemy n operacji na każdy pobrany element danych. Pozwala to na efektywne wykorzystanie pamięci podręcznej (cache) procesora i osiągnięcie wydajności bliskiej teoretycznemu maksimum (GFLOPS) procesora. Dlatego nowoczesne algorytmy (np. w bibliotece LAPACK) są konstruowane tak, aby jak najczęściej korzystać z funkcji BLAS Level 3.

23. Wyjaśnij całkowanie Monte Carlo metodą orzeł-reszka

Całkowanie Monte Carlo to metoda probabilistyczna obliczania całek oznaczonych, która opiera się na interpretacji geometrycznej całki jako pola powierzchni. Metoda “orzeł-reszka” (znana też jako metoda *hit-or-miss*) polega na losowaniu punktów wewnątrz prostokąta otaczającego wykres funkcji i sprawdzaniu, jaka frakcja tych punktów trafiła w obszar pod krzywą.

Załóżmy, że chcemy obliczyć całkę $\int_a^b f(x)dx$, gdzie $0 \leq f(x) \leq M$. Zamykamy wykres funkcji w prostokącie o podstawie $[a, b]$ i wysokości M . Pole tego prostokąta wynosi $S_{rect} = (b - a)M$. Następnie generujemy N losowych par liczb (x_i, y_i) , gdzie x_i jest losowane jednostajnie z $[a, b]$, a y_i z $[0, M]$. Dla każdego punktu sprawdzamy warunek: czy $y_i \leq f(x_i)$? Jeśli tak, to punkt leży pod wykresem (sukces). Niech K będzie liczbą punktów, które spełniły warunek. Wtedy przybliżona wartość całki wynosi:

$$I \approx S_{rect} \cdot \frac{K}{N}$$

Metoda ta jest bardzo prosta w implementacji i łatwo rozszerzalna na całki wielowymiarowe, gdzie metody deterministyczne zawodzą (“klątwa wymiarowości”). Jej główną wadą jest powolna zbieżność – błąd maleje proporcjonalnie do $1/\sqrt{N}$, co oznacza, że aby zwiększyć dokładność 10-krotnie, trzeba zwiększyć liczbę losowań 100-krotnie.

24. Na czym polega FFT - szybka transformata Fouriera; przedstaw algorytm, wyznacz jego złożoność obliczeniową

Dyskretna Transformata Fouriera (DFT) przekształca sygnał dyskretny z dziedziny czasu do dziedziny częstotliwości. Bezpośrednie obliczenie DFT dla wektora N -elementowego wymaga pomnożenia go przez macierz Fouriera, co wiąże się z wykonaniem N^2 mnożeń zespolonych. Szybka Transformata Fouriera (FFT) to algorytm, który oblicza ten sam wynik znacznie szybciej, wykorzystując symetrię i okresowość zespolonych pierwiastków z jedności. Najpopularniejszym wariantem jest algorytm Cooleya-tukeya typu “dziel i rządź”.

Idea FFT polega na rekurencyjnym podziale problemu o rozmiarze N na dwa mniejsze problemy o rozmiarze $N/2$: jeden dla próbek o indeksach parzystych, a drugi dla nieparzystych. Wyniki tych dwóch mniejszych transformat łączy się następnie za pomocą tzw. operacji motylkowych (ang. *butterfly operations*), aby uzyskać pełną transformację. Warunkiem najprostszej wersji algorytmu jest to, aby N było potęgą dwójki.

Złożoność obliczeniowa: W każdym kroku rekurencji wykonujemy $O(N)$ operacji łączenia. Ponieważ dzielimy problem na pół, głębokość rekurencji wynosi $\log_2 N$. Całkowita złożoność wynosi zatem $O(N \log N)$. Jest to ogromna redukcja w porównaniu do $O(N^2)$ dla klasycznego DFT, co czyni FFT jednym z najważniejszych algorytmów w przetwarzaniu sygnałów.

25. Porównaj znajdowanie minimum funkcji: metodą „złotego podziału” i metodą Brenta

Metoda złotego podziału jest odpowiednikiem metody bisekcji dla zadań optymalizacji (szukania minimum). Jest to metoda przedziałowa, która systematycznie zawęża obszar poszukiwań, opierając się wyłącznie na wartościach funkcji. Kluczową cechą jest podział odcinka w proporcji opartej na złotej liczbie $\phi \approx 0.618$, co pozwala na ponowne wykorzystanie jednego z punktów próbnych w kolejnej iteracji. Metoda ta gwarantuje znalezienie minimum dla funkcji unimodalnych, ale jej zbieżność jest liniowa (powolna).

Metoda Brenta jest algorytmem hybrydowym, łączącym niezawodność metody złotego podziału z szybkością interpolacji kwadratowej. Gdy funkcja zachowuje się regularnie, algorytm używa metody wstawiania odwrotnego paraboli (dopasowuje parabolę do trzech punktów i skacze do jej minimum), co zapewnia nadliniową zbieżność (rzęd ok. 1.324). Jeśli jednak krok paraboliczny jest niepewny, algorytm przełącza się na metodę złotego podziału. Dzięki temu metoda Brenta jest uznawana za standard w bibliotekach numerycznych (np. `gsl_min_fminimizer_brent`), oferując szybkość tam, gdzie to możliwe, i bezpieczeństwo tam, gdzie to konieczne.

26. Wyjaśnij dlaczego do konstruowania wydajnych algorytmów obliczeniowych konieczna jest znajomość architektur komputerowych

Współczesne procesory nie wykonują wszystkich operacji w takim samym czasie, a dostęp do pamięci jest znacznie wolniejszy niż same obliczenia. Dlatego wydajność algorytmu zależy nie tylko od jego złożoności matematycznej (liczby operacji), ale w ogromnym stopniu od sposobu, w jaki wykorzystuje on zasoby sprzętowe. Kluczowym elementem jest hierarchia pamięci (rejstry, cache L1/L2/L3, RAM). Algorytmy muszą być projektowane tak, aby maksymalizować lokalność danych – czyli przetwarzać dane, które już znajdują się w szybkiej pamięci podręcznej (cache), zamiast ciągle pobierać je z wolnej pamięci głównej. Przykładem jest mnożenie macierzy metodą blokową, która jest znacznie szybsza od metody naiwnej, mimo wykonania tej samej liczby mnożeń.

Kolejnym aspektem jest przetwarzanie wektorowe (instrukcje SIMD – Single Instruction, Multiple Data). Nowoczesne procesory potrafią wykonać tę samą operację (np. dodawanie) na wielu parach liczb jednocześnie w jednym cyklu zegara. Algorytm, który przetwarza dane w sposób sekwencyjny i zależny (gdzie wynik jednej operacji jest potrzebny do następnej), nie wykorzysta tego potencjału. Znajomość architektury pozwala również na wykorzystanie potokowości (pipelining) procesora oraz unikanie kosztownych instrukcji warunkowych, które „czyszczą” potok instrukcji (branch misprediction). Bez uwzględnienia tych czynników, nawet teoretycznie szybki algorytm może działać wolno w praktyce.

27. Przedstaw przykłady ilustrujące dążenie do redukcji złożoności obliczeniowej algorytmów obliczeniowych

Redukcja złożoności obliczeniowej jest głównym celem inżynierii algorytmicznej, pozwalającym na rozwiązywanie problemów o rozmiarach niedostępnych dla metod naiwnych. Klasycznym i najbardziej spektakularnym przykładem jest **Szybka Transformata Fouriera (FFT)**. Obliczenie Dyskretnej Transformaty Fouriera (DFT) bezpośrednio z definicji (jako mnożenie macierzy przez wektor) wymaga $O(N^2)$ operacji zespolonych. Algorytm FFT (Cooleya-tukeya), wykorzystujący strategię „dziel i rządź”, redukuje tę złożoność do $O(N \log N)$. Dla $N = 1000$ różnica jest stukrotna, ale dla miliona próbek FFT jest miliony razy szybsza, co umożliwia przetwarzanie sygnałów w czasie rzeczywistym.

Innym przykładem jest rozwiązywanie układów równań liniowych dla macierzy specjalnych. Ogólna metoda eliminacji Gaussa ma złożoność $O(N^3)$. Jeśli jednak wiemy, że macierz jest trójdzielna (ma niezerowe elementy tylko na głównej przekątnej i dwóch sąsiednich), możemy zastosować algorytm Thomasa, który ma złożoność liniową $O(N)$. Podobnie, algorytm Strassena do mnożenia macierzy redukuje złożoność z klasycznego $O(N^3)$ do około $O(N^{2.807})$, co przy bardzo dużych macierzach daje znaczący zysk czasowy.

28. Objaśnij miejsce metod komputerowych (teoria, eksperyment, symulacja komputerowa)

Tradycyjnie nauka opierała się na dwóch filarach: **teorii** i **eksperymentie**. Teoria zajmuje się budowaniem modeli matematycznych i wyprowadzaniem praw opisujących zjawiska (np. równania Naviera-Stokesa w mechanice płynów). Eksperyment służy do weryfikacji tych teorii poprzez obserwację rzeczywistości. Jednak współczesna nauka wprowadziła trzeci, równorzędny filar: **symulację komputerową (metody numeryczne)**.

Metody komputerowe wypełniają lukę między teorią a eksperymentem. Wiele modeli teoretycznych (np. prognozowanie pogody, dynamika galaktyk) jest zbyt skomplikowanych, by rozwiązać je analitycznie (na kartce papieru). Z drugiej strony, wiele eksperymentów jest zbyt kosztownych (testy zderzeniowe samochodów), niebezpiecznych (wybuchy jądrowe), zbyt powolnych (ewolucja gwiazd) lub niemożliwych do przeprowadzenia

(badanie wnętrza Ziemi). Symulacja komputerowa pozwala na „wirtualny eksperyment” – numeryczne rozwiązanie równań teorii w celu przewidzenia zachowania układu. Dzięki temu metody komputerowe stały się niezbędnym narzędziem we współczesnej inżynierii, fizyce, chemii i ekonomii.

part 2

1. Wyprowadź wzór na błąd względny reprezentacji zmiennoprzecinkowej

Reprezentacja zmiennoprzecinkowa liczby rzeczywistej x w komputerze wiąże się z koniecznością zaokrąglania jej do najbliższej liczby maszynowej $fl(x)$. Jeśli przyjmiemy standardowy model arytmetyki, w którym liczba zapisywana jest jako $x = \pm m \cdot \beta^c$, gdzie m to mantysa, β to podstawa systemu (zazwyczaj 2), a c to cecha, to mantysa w komputerze ma ograniczoną długość t cyfr. Błąd bezwzględny zaokrąglania, czyli różnica między liczbą rzeczywistą a jej reprezentacją maszynową, jest ograniczona przez połowę odstepu między liczbami maszynowymi w danym przedziale. Maksymalny błąd bezwzględny wynosi $|x - fl(x)| \leq \frac{1}{2}\beta^{c-t}$.

Błąd względny definiujemy jako stosunek błędu bezwzględnego do wartości liczby: $\delta = \frac{|x - fl(x)|}{|x|}$. Podstawiając ograniczenie na błąd bezwzględny oraz wiedząc, że znormalizowana mantysa spełnia warunek $\beta^{-1} \leq m < 1$, a zatem $|x| \geq \beta^{c-1}$, otrzymujemy oszacowanie błędu względnego. Dzieląc nierówność przez $|x|$, uzyskujemy $\frac{|x - fl(x)|}{|x|} \leq \frac{\frac{1}{2}\beta^{c-t}}{\beta^{c-1}} = \frac{1}{2}\beta^{1-t}$. Wartość $\epsilon = \frac{1}{2}\beta^{1-t}$ nazywamy epsilonem maszynowym (dla zaokrąglania). Oznacza to, że reprezentacja zmiennoprzecinkowa liczby x może być zapisana jako $fl(x) = x(1 + \delta)$, gdzie moduł błędu względnego $|\delta|$ nie przekracza epsilona maszynowego ϵ .

2. Wyprowadź wzór na interpolację metodą Lagrange’a

Problem interpolacji polega na znalezieniu wielomianu $L_n(x)$ stopnia co najwyżej n , który w zadanych węzłach $x_0, x_1, \dots, x_n$ przyjmuje zadane wartości $y_0, y_1, \dots, y_n$. W metodzie Lagrange’a wielomian ten konstruujemy jako kombinację liniową wielomianów bazowych $l_i(x)$. Kluczową własnością wielomianów bazowych jest to, że w i -tym węźle przyjmują wartość 1, a we wszystkich pozostałych węzłach wartość 0. Matematycznie zapisujemy to za pomocą delty Kroneckera: $l_i(x_j) = \delta_{ij}$.

Aby spełnić ten warunek, wielomian bazowy $l_i(x)$ musi mieć pierwiastki we wszystkich węzłach x_j dla $j \neq i$. Postać takiego wielomianu to iloczyn czynników $(x - x_j)$. Aby znormalizować ten wielomian tak, by w punkcie x_i wynosił dokładnie 1, dzielimy go przez wartość tego iloczynu w punkcie x_i . Ostateczny wzór na i -ty wielomian bazowy Lagrange’a przyjmuje postać $l_i(x) = \prod_{j=0, j \neq i}^n \frac{x - x_j}{x_i - x_j}$. Poszukiwany wielomian interpolacyjny jest sumą ważoną tych wielomianów bazowych, gdzie wagami są wartości funkcji w węzłach: $L_n(x) = \sum_{i=0}^n y_i l_i(x)$. Taka konstrukcja gwarantuje, że dla każdego węzła x_k , wielomian przyjmie wartość y_k , ponieważ w sumie tylko jeden składnik (gdzie $i = k$) będzie niezerowy i równy $y_k \cdot 1$.

3. Wyprowadź wzór na aproksymację średniokwadratową wielomianem uogólnionym

Aproksymacja średniokwadratowa polega na znalezieniu takiej funkcji $P(x)$ (będącej kombinacją liniową funkcji bazowych $\phi_0, \phi_1, \dots, \phi_n$), która minimalizuje błąd średniokwadratowy względem aproksymowanej funkcji $f(x)$ na przedziale $[a, b]$ z wagą $\rho(x)$. Błąd ten definiujemy jako funkcjonal $S = \int_a^b [f(x) - \sum_{k=0}^n c_k \phi_k(x)]^2 \rho(x) dx$. Aby zminimalizować S , obliczamy pochodne cząstkowe względem współczynników c_k i przyrównujemy je do zera. Prowadzi to do układu równań normalnych.

Sytuacja ulega znacznemu uproszczeniu, gdy jako bazę wybierzemy wielomiany ortogonalne, czyli takie, dla których iloczyn skalarny $\langle \phi_i, \phi_j \rangle = \int_a^b \phi_i(x) \phi_j(x) \rho(x) dx$ wynosi 0 dla $i \neq j$. Wówczas macierz układu równań normalnych staje się macierzą diagonalną. Dzięki temu każdy współczynnik c_k można wyliczyć niezależnie od pozostałych, korzystając ze wzoru $c_k = \frac{\langle f, \phi_k \rangle}{\langle \phi_k, \phi_k \rangle}$. Ostatecznie wzór na najlepszy wielomian aproksymujący w sensie średniokwadratowym to $P(x) = \sum_{k=0}^n \frac{\int_a^b f(x) \phi_k(x) \rho(x) dx}{\int_a^b \phi_k^2(x) \rho(x) dx} \phi_k(x)$.

4. Wyprowadź wzory na kwadratury elementarne: trapezów i prostokątów

Kwadratury to metody numerycznego obliczania całek oznaczonych $\int_a^b f(x)dx$. Kwadratura prostokątów opiera się na przybliżeniu funkcji podcałkowej stałą wartością, równą wartości funkcji w środku przedziału $c = \frac{a+b}{2}$. Geometrycznie oznacza to zastąpienie pola pod krzywą polem prostokąta o podstawie $b - a$ i wysokości $f(c)$. Wzór na kwadraturę prostokątów to $I \approx (b - a)f(\frac{a+b}{2})$. Rząd błędu tej metody wynosi $O(h^3)$ dla pojedynczego przedziału.

Kwadratura trapezów aproksymuje funkcję podcałkową wielomianem interpolacyjnym pierwszego stopnia (prostą) przechodzącą przez punkty $(a, f(a))$ i $(b, f(b))$. Pole pod wykresem funkcji zastępujemy polem trapezu o wysokości $h = b - a$ i podstawach równych wartościom funkcji na krańcach przedziału. Wzór wyprowadzamy z pola trapezu: $I \approx \frac{b-a}{2}[f(a) + f(b)]$. Chociaż rząd błędu lokalnego wynosi również $O(h^3)$, współczynnik błędu jest zazwyczaj większy niż w metodzie prostokątów.

5. Wyprowadź wzór na kwadraturę Simpsona

Kwadratura Simpsona powstaje przez zastąpienie funkcji podcałkowej $f(x)$ na przedziale $[a, b]$ wielomianem interpolacyjnym drugiego stopnia (parabolą), który przechodzi przez trzy punkty: lewy kraniec a , środek przedziału $c = \frac{a+b}{2}$ oraz prawy kraniec b . Oznaczmy krok całkowania jako $h = \frac{b-a}{2}$. Wzór interpolacyjny Lagrange'a dla tych trzech węzłów całkujemy w granicach od a do b .

Po wykonaniu całkowania wielomianu kwadratowego otrzymujemy wzór, który jest średnią ważoną wartości funkcji w węzłach. Wagi wynoszą odpowiednio 1 dla krańców i 4 dla środka. Ostateczny wzór na kwadraturę Simpsona to: $\int_a^b f(x)dx \approx \frac{b-a}{6} [f(a) + 4f(\frac{a+b}{2}) + f(b)]$ lub zapisując przy użyciu kroku h : $I \approx \frac{h}{3}[f(a) + 4f(c) + f(b)]$. Metoda ta jest dokładna dla wielomianów stopnia co najwyżej 3, a jej lokalny błąd jest rzędu $O(h^5)$.

6. Wyprowadź wzór na metodę Newtona-Raphsona i jej rząd zbieżności

Metodę Newtona-Raphsona wyprowadzamy, rozwijając funkcję $f(x)$ w szereg Taylora wokół bieżącego przybliżenia x_n . Mamy: $f(x) \approx f(x_n) + f'(x_n)(x - x_n)$. Ponieważ szukamy miejsca zerowego, przyrównujemy to przybliżenie do zera: $f(x_n) + f'(x_n)(x - x_n) = 0$. Rozwiązując to równanie liniowe względem x , otrzymujemy wzór iteracyjny na kolejne przybliżenie: $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$.

Aby określić rząd zbieżności, definiujemy błąd w n -tym kroku jako $\epsilon_n = x_n - \alpha$, gdzie α to dokładny pierwiastek ($f(\alpha) = 0$). Rozwijając funkcję $f(\alpha)$ w szereg Taylora wokół x_n , otrzymujemy $0 = f(\alpha) = f(x_n) + f'(x_n)(\alpha - x_n) + \frac{1}{2}f''(\xi)(\alpha - x_n)^2$. Dzieląc przez $f'(x_n)$ i wykorzystując definicję metody Newtona, dochodzimy do relacji $\epsilon_{n+1} \approx C \cdot \epsilon_n^2$, gdzie $C = \frac{f''(\alpha)}{2f'(\alpha)}$. Zależność ta pokazuje, że błąd w następnym kroku jest proporcjonalny do kwadratu błędu w kroku poprzednim, co oznacza, że metoda Newtona-Raphsona ma kwadratowy rząd zbieżności (rząd 2).

7. Wyprowadź złożoność obliczeniową metody Gaussa rozwiązywania układów równań liniowych

Metoda eliminacji Gaussa składa się z dwóch etapów: eliminacji w przód (sprowadzanie macierzy do postaci trójkątnej górnej) oraz podstawiania wstecz. Koszt obliczeniowy jest zdominowany przez etap eliminacji. Dla macierzy o wymiarze $n \times n$, w i -tym kroku eliminacji (gdzie i zmienia się od 1 do $n - 1$), musimy wyzerować elementy pod przekątną w kolumnie i . Dla każdego wiersza j poniżej przekątnej ($j > i$), obliczamy mnożnik, a następnie odejmujemy od wiersza j wiersz i przemnożony przez ten mnożnik. Operacja ta obejmuje około $n - i$ mnożeń i dodawań.

Sumując liczbę operacji (głównie mnożeń i dzieleni) dla wszystkich kroków, otrzymujemy sumę postaci $\sum_{i=1}^{n-1} (n - i)(n - i + 1) \approx \sum_{k=1}^n k^2$. Znając wzór na sumę kwadratów liczb naturalnych, wiemy, że wynosi ona $\frac{n(n+1)(2n+1)}{6}$. Dla dużych n dominującym wyrazem jest $\frac{2n^3}{6} = \frac{n^3}{3}$. Etap podstawiania wstecz wymaga około $O(n^2)$ operacji, co jest pomijalne przy eliminacji. Zatem asymptotyczna złożoność obliczeniowa metody eliminacji Gaussa wynosi $O(n^3)$.